Poisoning Attacks and Data Sanitization Mitigations for Machine Learning Models in Network Intrusion Detection Systems

Sridhar Venkatesan, Harshvardhan Sikka, Rauf Izmailov, Ritu Chadha
Peraton Labs Inc.
Basking Ridge, NJ, USA
{*svenkatesan, harsh.sikka, rizmailov, rchadha*}*@peratonlabs.com*

Alina Oprea
Northeastern University
Boston, MA, USA
a.oprea@northeastern.edu

Michael J. De Lucia
DEVCOM Army Research Laboratory
Aberdeen Proving Ground, MD
michael.j.delucia2.civ@mail.mil

MILCOM 2021 - 2021 IEEE Military Communications Conference (MILCOM) | 978-1-6654-3956-5/21/$31.00 ©2021 IEEE | DOI: 10.1109/MILCOM52596.2021.9652916

***Abstract*—Among many application domains of machine learning in real-world settings, cyber security can benefit from more automated techniques to combat sophisticated adversaries. Modern network intrusion detection systems leverage machine learning models on network logs to proactively detect cyber attacks. However, the risk of adversarial attacks against machine learning used in these cyber settings is not fully explored. In this paper, we investigate poisoning attacks at training time against machine learning models in constrained cyber environments such as network intrusion detection; we also explore mitigations of such attacks based on training data sanitization. We consider the setting of poisoning availability attacks, in which an attacker can insert a set of poisoned samples at training time with the goal of degrading the accuracy of the deployed model. We design a white-box, realizable poisoning attack that reduced the original model accuracy from 95% to less than 50% by generating mislabeled samples in close vicinity of a selected subset of training points. We also propose a novel Nested Training method as a defense against these attacks. Our defense includes a diversified ensemble of classifiers, each trained on a different subset of the training set. We use the disagreement of the classifiers' predictions as a data sanitization method, and show that an ensemble of 10 SVM classifiers is resilient to a large fraction of poisoning samples, up to 30% of the training data.**

***Index Terms*—Machine learning, Adversarial machine learning, Network intrusion detection.**

1. Introduction

There has been an increasing deployment of Network Intrusion Detection Systems (NIDS) to detect adversarial activities within a network. Machine Learning (ML) algorithms are the core of a NIDS's decision process and have become a new attack surface for adversaries. Adversarial ML is a research area that studies the vulnerabilities of ML against a range of threats, and proposes mitigations against these risks. Most of attacks against ML can be broadly classified as *evasion* and *poisoning* attacks; the former occur at inference time, while the latter occur when training a model.

Currently, most of adversarial ML research is on *evasion attacks* at inference time [1], [2]. We argue that to safely deploy ML in critical applications, an important threat that needs to be addressed is that of *poisoning attacks* at training time. In a poisoning attack, the adversary can modify a limited fraction of labels in existing training data samples or add a limited fraction of mislabeled new ones into the training such that an ML trained on that set behaves according to the adversary's objective. According to a Microsoft survey [3], ML-deploying companies considered poisoning attacks as their *top concern*. Moreover, as supply chain vulnerabilities gained a lot of attention with the recent SolarWinds attacks [4], it becomes extremely important to understand the vulnerabilities that can be introduced during the training process and secure the entire ML supply chain.

Developing ML models for NIDS requires a training dataset composed of benign and malicious network traffic logs. While benign traffic can be collected from the environment on which NIDS is deployed, attack traffic is primarily collected from different third-party sources (e.g., those listed in [5]). Attack traffic is collected by running a malware in a sandboxed environment and labeling all packets generated by the malware as attack. A malign third-party could modify the generated traffic or mislabel samples to degrade the performance of the ML models trained on the poisoned dataset. Increasing the false positive rate of the trained detector increases the workload on an already resource-constrained Security Operations Center (SOC) and subsequently, diverts the analysts attention from addressing critical issues.

Poisoning attacks can be classified according to the attacker's objective as: *availability attacks* in which the model's prediction is impacted indiscriminately [6], [7]; *targeted attacks* in which the model mis-classifies a small set of targeted points [8], [9]; and *backdoor attacks* in which the model mis-classifies samples including a backdoor pattern [10]. Recently, subpopulation poisoning attacks have been proposed as an interpolation between availability and targeted attacks [11].

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 11:09:59 UTC from IEEE Xplore. Restrictions apply.

While there has been a large body of work studying poisoning attacks in unconstrained continuous domains such as computer vision (e.g., [6], [10]), tabular and text data ([7], [11]), existing work on poisoning attacks and defenses in cyber domain have been limited to specific applications. These include worm signature generation [12], [13], spam filters [14], DoS attack detection [15], and behavioral malware clustering [16]. However, results of these studies usually do not generalize to other settings, although [17] describe a mechanisms for model-agnostic backdoor attacks using explainability techniques.

One of the challenges of poisoning a cyber dataset is that the poisoned samples should be *realizable*. As noted above, third-party sources publish raw packet captures and thus, an adversary's modifications to say, a packet, are constrained to the valid values that a *well-formed* packet can assume (e.g., maintaining header ordering, allowing only valid combinations of TCP flags). Furthermore, the resulting poisoned samples should *affect* the ML model's performance according to the adversary's objective.

In this paper, we first study poisoning availability attacks against a network scanning classifier that detects scan traffic by processing network flow logs. For this setting, we start by showing that a state-of-the-art poisoning availability attack by Biggio et al. [6] is not successful against a trained network scanning classifier. We then propose a new technique for executing a poisoning availability attack in which a limited number of poisoned samples that *look similar* to the high-confidence training samples are injected into the training dataset. The rationale is that, since high-confidence training samples influence a ML model's decision, injecting poisoned samples close (in the feature space) to the high-confidence data affects the training process by effectively *propagating* the labels of the poisoned samples to their influential neighbors and thus, significantly degrading the resulting model's performance. This technique also ensures that the attack is realizable by constraining the modifications (to generate poisoned samples) to only allowable values that a network flow can assume.

We evaluated the attacks for an adversarial budget (i.e., the ratio of the maliciously mislabeled samples to the size of the original training dataset) of up to 20% poisoning rate, by varying the distance (in the feature space) between the injected poisoned samples and the targeted high-confidence samples. By placing poisoned samples close to 10% of high-confidence points, the attack increases the classification error from 5% to more than 50%.

We also address the well-known problem of defending against poisoning attacks. Several defenses against availability attacks are available such as using robust statistics techniques for training linear regression models [7] and also, against backdoor attacks in computer vision [18], [19]; however, they are not directly applicable for defending general classifiers against poisoning availability attacks. In this paper, we propose an ensemble-based approach for identifying those poisoned samples that can affect the ML model performance and sanitize the training data by removing them.

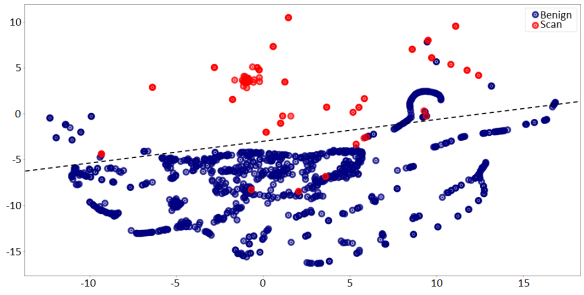


Figure 1: t-distributed stochastic neighbor embedding (t-SNE) plot of all samples by class in the scanning dataset.

To identify poisoned samples, we train a set of diversified models, each based on a different subset of the original training dataset. Since the number of poisoned samples is relatively small (the attack budget of the adversary is limited), their distribution among different training subsets will vary significantly. Indeed, while computing a reliable statistic requires sufficiently large sample sizes, here we leverage a converse property – if the sample size is small, then statistics will *not* be reliable, i.e., it will vary. As a result, our models treat poisoning points (and, more generally, anomalous points) in a statistically divergent manner since each of the training subsets will contain a highly varied number of poisoned samples thus enabling their detection via levels of disagreement among individual models. For detection of poisoned (and anomalous) data, the ensemble disagreement is higher on poisoned points (out-of-distribution) compared to legitimate points, which follow the normal data distribution. We show that with just 10 diversified SVM models in the ensemble, and 30% poisoning rate, we remove almost all the poisoned points (along with a small number of benign ones), and, as a result, completely restore the original model accuracy.

The rest of paper is organized as follows. Section 2 describes a network scanning classifier and different approaches for poisoning availability attacks against it. Section 3 describes the Nested Training approach for sanitizing a poisoned datasets and finally, Section 4 provides conclusions and future work.

2. Poisoning Attacks Against ML in NIDS

2.1. Network Scanning Dataset

For our experiments, we considered a network scanning detector that was trained on an existing network scanning dataset. The dataset, used by researchers in [20], was composed of scanning flows generated by *Nmap* with different tool configurations (e.g., stealthy and slow scan, aggressive and fast scan) and benign flows that were produced by a benign traffic generation tool. Each record in the dataset contained the following features for a flow: duration, number of packets sent and received, number of bytes sent and received, total number of packets and bytes, and TCP flags summary in both directions.

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 11:09:59 UTC from IEEE Xplore. Restrictions apply.

There were over 15,000 network flow records in the dataset among which 2,300 records were scanning data and the rest were benign data. Fig. 1 visualizes the similarity between the data points using a t-distributed stochastic neighbor embedding (t-SNE) dimension reduction technique to place the feature space onto a 2-dimensional space. As Fig. 1 shows, the data is linearly separable, so we trained a linear SVM detector model with the penalty hyper-parameter set to 1; the accuracy of the resulting classifier was more than 95%. We observed that the features based on total bytes and the duration had the most influence in the classifier's decision. For the rest of the paper, we consider linear SVM models for detecting scanning flows.

2.2. Threat Model

The adversary's goal is to significantly reduce the accuracy of a network scanning detector by poisoning its training dataset. In this work, we assume that the adversary controls a small fraction of the traffic data (and its labels) that was used to train the detector. Furthermore, as mentioned in the introduction section, any modifications to the traffic data that the adversary controls should respect the constraints imposed by the physical network and the network protocols (e.g., ensuring the maximum number of bytes in a packet does not exceed 1460). These modifications can be achieved through multiple ways: for instance, to modify scan data, an adversary could directly modify the contents of the scan packets or modify Nmap's port probing behavior.

2.3. Poisoning of Network Scanning Dataset

In this section we study the effect of poisoning by first considering a state-of-the-art attack used in image domain and then describe a new technique for poisoning the network scanning dataset that targets high-confidence points.

2.3.1. State-of-the-art attack performance. We considered a gradient ascent method proposed by Biggio et al. against SVMs [6]. The algorithm starts with a point in the attacked class and updates the point by computing the gradient of the loss function in the direction of the attacked class. The proposed attack significantly increased the validation and testing errors of a SVM classifier trained on MNIST dataset. To evaluate the effectiveness of the algorithm against the network scanning dataset, we used an existing implementation of the attack available in [21]. Although we did not apply any additional realizability constraints to poisoned samples obtained from this method, we observed that even with a 20% poisoning rate, the error rate was less than 10%. The robustness of the classifier against the attack was due to two reasons: (i) unlike the MNIST dataset, the benign and scan distributions were far apart in the geometric input space, and (ii) the scan flows in the dataset had a much smaller variance than the benign distribution. As a result, within a few iterations, the gradients computed on the loss function converged and did not increase the loss significantly.

2.3.2. High confidence data poisoning. Since the attack in [6] was not successful against network scan dataset, we designed a novel attack approach based on targeting high confidence (influential) data points. In this attack, we augment training data set with poisoned samples located closely (in the feature space) to the high-confidence samples. Specifically, we add scan-labeled samples that looked similar (i.e., close in the feature space) to the high-confidence benign-labeled samples. As a result of this poisoning strategy, the training process was affected in a way that was similar to the actual label change, i.e., if a few poisoning samples with label A are injected close to a sample with label B, the training process will treat that cloud of points as point(s) with label A. We evaluated the performance of these attacks by varying (i) the number of added poisoned data points, and (ii) their proximity to the targeted samples.

Additionally, we also included realizability constraints on the poisoning samples following the approach in [22]: we allowed only 3 features to be modifiable, i.e., flow duration, # packets from sender (S) to receiver (R), and # bytes from S to R. We also included feature dependencies such as *total number of packets* should be the sum of the *# pkts from S to R* and *# pkts from R to S* and similarly, for the bytes features. Finally, we also included boundary constraints onto which the poisoned points were projected. Specifically, we ensured that flow duration, # pkts and # bytes are non-negative and also, upper bounding the *# bytes sent* to *# pkts sent* * 1460 (i.e., max bytes in a packet). While these realizability constraints were applied on the feature space, scan tools or the scan traffic can be modified to generate such feature vectors.

We examined the effect of distance between the poisoned samples and the attacked high confidence samples on the classifier's performance. We added 2 poisoned points around the highest confidence benign points, sampled from increasing subsets of training data. We evaluate this under different distances by sampling poisoned points from an L2-norm ball of different radii. Fig. 2 demonstrates results for this experiment for two L2-norm sampling scenarios.

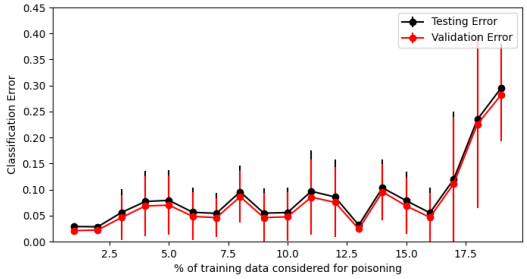
As expected, placing poisoned points closer (based on L2-norm) to the high confidence points results in higher error rates; those error rates increase with larger portions of the training data being considered for poisoning. However, at larger distances, the effect of increasing the training data subset for poisoning has a smaller impact, as can be seen in the differing behavior in Fig. 2a and Fig. 2b.

We also compare different numbers of poisoned points added sampled from increasing subsets of training data with a fixed distance. We find that, for a given percentage of training data considered for poisoning, the classifier performance deteriorates with more poisoned points added to the data, as seen in Fig. 3.

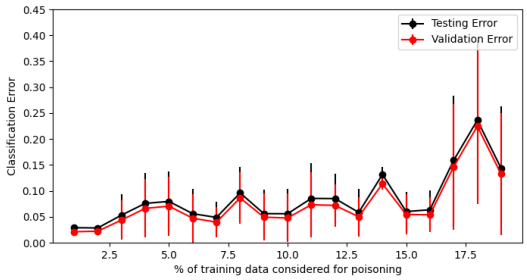
3. Data Sanitization

Our approach towards creating a set of *diversified* classifiers is based on training the base classifiers on random subsets of the original training dataset. The reasoning for that

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 11:09:59 UTC from IEEE Xplore. Restrictions apply.



(a) Poisoning points sampled from a L2-norm ball of radius 0.1.



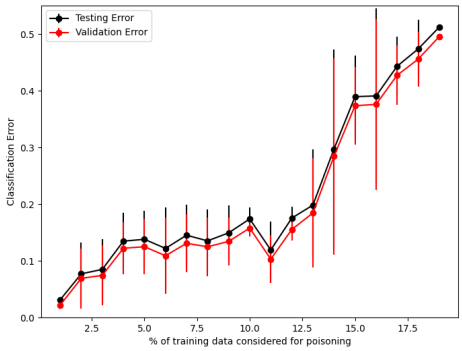
(b) Poisoning points sampled from a L2-norm ball of radius 0.6

Figure 2: Classification errors when 2 poisoning points are added at different distances from the high confidence points.

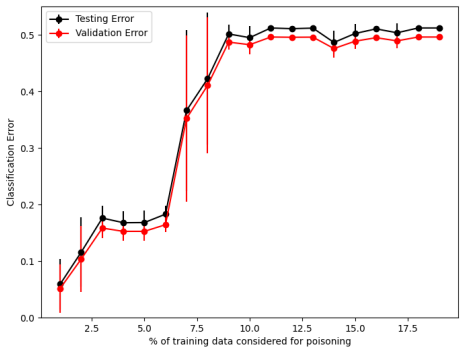
diversification method is as follows. Since a relatively small portion of the training data have been actually poisoned, these individual random subsets will contain rather varying numbers of poisoned data, which would, in turn, vary both the effects of poisoned data on the classifier training and the decisions of the trained classifiers on poisoned data.

To identify poisoned points in the cyber dataset, we considered an ensemble of 10 linear SVM classifiers, each trained on a random subset of the training data. Since poisoned points comprise a small portion of the training data, the classifiers are expected to exhibit *larger* disagreements on poisoned points and *smaller* disagreements on the clean points. To measure the disagreement, through our experiments, we observed that computing the standard deviation of the logits was sufficient to obtain dissimilar *disagreement distributions* for clean and poisoning points. Figs. 4 depict the standard deviations in the scores of the 10 classifiers.

Another factor that governs the dissimilarity in the disagreement distributions is the size of the training subset used for learning the classifiers. As shown in Fig. 4, ensembles trained on smaller random subsets result in higher disagreements among the classifiers' scores. However, smaller training sizes would result in issues due to under-sampling from a class and as a result, the classifiers may have a higher disagreement on clean points as well. Fig. 5 captures the KL-divergence in the disagreement distributions as a function of the training size; the higher is the KL-divergence, the higher is the discernability between clean and poisoned points.



(a) Adding 3 points



(b) Adding 7 points

Figure 3: Classification errors as a function of the number of poisoning points around the high confidence points.

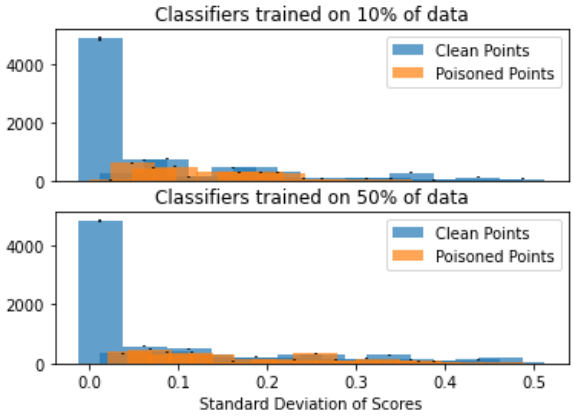


Figure 4: Histogram of standard deviations of scores from 10 SVM classifier trained on random subset of training data

3.1. Nested Training for Data Sanitization

From Fig. 4, it can be seen that, while disagreement levels of diversified classifiers on poisoned and clean points are quite different, there is still a significant overlap in their disagreement distributions. Consequently, although the mode of the disagreement distributions for clean points is less than that of poisoned points, simple thresholding

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 11:09:59 UTC from IEEE Xplore. Restrictions apply.

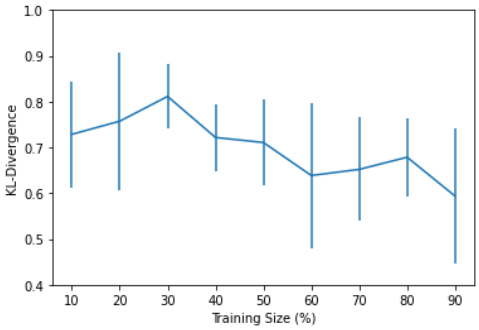


Figure 5: KL-Divergence b/w disagreement distributions of logits for clean and poisoned points averaged over 10 runs.



Figure 6: Performance of Nested Training approach when 2 poisoning points are added around 17% of the highest confidence training points.

schemes that discard points based on large disagreements (i.e., standard deviations) either significantly reduce the size of the resulting training set or retain a large portion of the poisoned points (depending on the training size). For instance, when the training size is 10% of the original dataset, such a simple thresholding scheme discarded all of the poisoned points with 50% of the clean points and increased the test error rate to 33%. Furthermore, due to the spread of the disagreement distributions, using clustering approaches (similar to [19]) was uninformative in identifying the poisoned points.

In order to sanitize a poisoned dataset, we developed a *Nested Training* approach, which leverages the *influence* of training points on the classification error rates towards deciding whether they should be used for training at all. The approach monitors the training and validation error rates of a classifier trained on sequential nested subsets of the training data – each containing previous ones – and uses the observed change in error rates as an indicator of emerging poisoned data influence. In order to compose the next subset, we augment the previous one with those points that cause minimum disagreement among the diversified classifiers.

An outline of our data sanitization approach using the nested training is provided in Algorithm 1. As a preprocessing step, it trains an ensemble of diversified classifiers as described above. It then computes the standard deviations in the scores obtained from the diversified classifiers for each training point in the dataset (line 1). Next, it orders the training points in increasing order of disagreement (line 2). The rationale is that since training points with small disagreements are potentially clean points, their inclusion to an existing training data should not change the error rate of a classifier too much. Furthermore, including such points at an early stage establishes a baseline for classification error.

At each iteration in lines 4 - 11, it expands the training data size by δ (we used δ as 10% of the original training size) and includes points with the smallest disagreement. Next, it trains a classifier on the new training data and computes the classification error. Finally, it monitors the increase in error rate (lines 8 - 10), and if it crosses a pre-defined threshold τ, it stops including more points and returns the latest composed training dataset – points not included in the dataset are considered to be potentially poisoned and thus discarded.

We compared the approach against a random strategy in which a classifier is trained on random subsets of training data of different size. From Fig. 6, it can be seen that when there are very few training points, the test and validation errors are high due to under-sampling. As more training points are considered, the error rate falls to very low values and stay there. But when more than 60% of the training dataset is considered, the poisoned points start affecting the classifier's performance with the sudden climb in the error rates; the resulting dataset of 60% of the original dataset is then used for the final classifier that provides the original performance. For the considered poisoning rate, we observed that less than 10% of the *hard* examples (i.e., samples with confidence in $[0.25, 0.75]$) were removed due to disagreement between classifiers within the ensemble.

Algorithm 1: Nested Training for Sanitization

Input: An ensemble of N base models $\{\mathcal{F}_i\}$, training size increments $\delta \in (0, 1)$, error tolerance $\tau \in (0, 1)$
Output: A subset of training dataset $\mathcal{D}^s_{tr}$
Data: Poisoned training data $\mathcal{D}_{tr}$ of size M, Validation data $\mathcal{D}_{val}$

1 Compute standard deviation in scores $\sigma_i \leftarrow \sigma(\mathcal{F}_0(x_i), ..., \mathcal{F}_N(x_i)), \forall (x_i, y_i) \in \mathcal{D}_{tr}$
2 Sort points $(x_i, y_i) \in \mathcal{D}_{tr}$ in increasing order of σ_i
3 Initialize $\Delta \leftarrow [\delta * M]$, $s \leftarrow \Delta$, $\mathcal{E}^{val}_{s'} \leftarrow 1$
4 **while** $s \leq M$ **do**
5 Compose new training dataset, $D^s_{tr} \leftarrow \bigcup_{y \in \mathcal{Y}} \bigcup_{i=1}^{s} (x_i, y)$
6 Train a base model $\mathcal{G}_s$ on D^s_{tr}
7 Compute classification error $\mathcal{E}^{val}_s$ of $\mathcal{G}_s$ on D_{val}
8 **if** $(\mathcal{E}^{val}_s - \mathcal{E}^{val}_{s'}) \geq \tau$ **then**
9 **break**
10 Update $\mathcal{E}^{val}_{s'} \leftarrow \mathcal{E}^{val}_s$
11 Increase training size, $s \leftarrow s + \Delta$
12 **return** $D^{s'}_{tr}$

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 11:09:59 UTC from IEEE Xplore. Restrictions apply.

4. Conclusions and Future Work

In this paper, we addressed the issue of poisoning attacks against NIDS by (i) devising a new realizable poisoning attack, and (ii) developing a novel Nested Training approach to sanitize data using ensemble learning. We first considered a network scanning detector and showed that it is robust against an existing state-of-the-art poisoning attack. We then devised an approach that introduced poisoned samples around high-confidence points and ensured realizability by subjecting the samples to a set of network constraints. Finally, we proposed a Nested Training approach that leverages the disagreements within an ensemble of classifiers to filter out potential poisoned samples.

In future work we plan to extend our diversification methods using an ensemble of feature subsets. We believe subsets of features might change the feature importance and introduce further challenges to adversarial crafted poisoned examples inserted into the training dataset. We also plan to introduce a two-stage ensemble to incorporate a soft labeling mechanism similiarly to [23].

Acknowledgments

This research was sponsored by the DEVCOM Army Research Laboratory and was accomplished under Cooperative Agreement Number W911NF-13-2-0045 (ARL Cyber Security CRA). The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the DEVCOM Army Research Laboratory or the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation here on.

References

[1] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, "Intriguing properties of neural networks," in *International Conference on Learning Representations*, 2014. [Online]. Available: http://arxiv.org/abs/1312.6199

[2] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *2017 ieee symposium on security and privacy (sp)*. IEEE, 2017, pp. 39–57.

[3] R. S. Siva Kumar, M. Nyström, J. Lambert, A. Marshall, M. Goertzel, A. Comissoneru, M. Swann, and S. Xia, "Adversarial machine learning-industry perspectives," in *2020 IEEE Security and Privacy Workshops (SPW)*, 2020, pp. 69–75.

[4] E. D. Wolff, K. M. Growley, M. G. Gruden, and et al., "Navigating the solarwinds supply chain attack," *The Procurement Lawyer*, vol. 56, no. 2, 2021.

[5] M. Ring *et al.*, "A survey of network-based intrusion detection data sets," *Computers & Security*, vol. 86, pp. 147–167, 2019.

[6] B. Biggio, B. Nelson, and P. Laskov, "Poisoning attacks against support vector machines," in *Proceedings of the 29th International Coference on International Conference on Machine Learning*, ser. ICML'12. Edinburgh, Scotland: Omnipress, Jun. 2012, pp. 1467–1474.

[7] M. Jagielski, A. Oprea, B. Biggio, C. Liu, C. Nita-Rotaru, and B. Li, "Manipulating Machine Learning: Poisoning Attacks and Countermeasures for Regression Learning," in *2018 IEEE Symposium on Security and Privacy (SP)*. San Francisco, CA: IEEE, May 2018.

[8] A. Shafahi, W. R. Huang, M. Najibi, O. Suciu, C. Studer, T. Dumitras, and T. Goldstein, "Poison Frogs! Targeted Clean-Label Poisoning Attacks on Neural Networks," in *Advances in Neural Information Processing Systems*, Apr. 2018.

[9] J. Geiping, L. Fowl, W. Ronny Huang, W. Czaja, G. Taylor, M. Moeller, and T. Goldstein, "Witches' Brew: Industrial scale data poisoning via gradient matching," in *9th International Conference on Learning Representations, ICLR 2021*, 2021.

[10] T. Gu, B. Dolan-Gavitt, and S. Garg, "BadNets: Identifying Vulnerabilities in the Machine Learning Model Supply Chain," *arXiv:1708.06733 [cs]*, Aug. 2017, arXiv: 1708.06733. [Online]. Available: http://arxiv.org/abs/1708.06733

[11] M. Jagielski, G. Severi, N. P. Harger, and A. Oprea, "Subpopulation data poisoning attacks," *ArXiv*, vol. abs/2006.14026, 2020.

[12] J. Newsome, B. Karp, and D. Song, "Paragraph: Thwarting signature learning by training maliciously," in *Recent Advances in Intrusion Detection*, ser. LNCS. Springer, 2006, pp. 81–105.

[13] R. Perdisci, D. Dagon, W. Lee, P. Fogla, and M. Sharif, "Misleading worm signature generators using deliberate noise injection," in *Proc. IEEE Security and Privacy Symposium*, ser. S&P, 2006.

[14] B. Nelson, M. Barreno, F. J. Chi, A. D. Joseph, B. I. Rubinstein, U. Saini, C. Sutton, J. Tygar, and K. Xia, "Exploiting machine learning to subvert your spam filter," in *Proc. First USENIX Workshop on Large-Scale Exploits and Emergent Threats*, ser. LEET, 2008.

[15] B. I. Rubinstein, B. Nelson, L. Huang, A. D. Joseph, S.-h. Lau, S. Rao, N. Taft, and J. D. Tygar, "ANTIDOTE: understanding and defending against poisoning of anomaly detectors," in *Proceedings of the 9th ACM SIGCOMM Internet Measurement Conference*, ser. IMC '09. New York, NY, USA: ACM, 2009, pp. 1–14.

[16] B. Biggio, K. Rieck, D. Ariu, C. Wressnegger, I. Corona, G. Giacinto, and F. Roli, "Poisoning behavioral malware clustering," in *Proceedings of the 2014 Workshop on Artificial Intelligent and Security Workshop - AISec '14*. Scottsdale, Arizona, USA: ACM Press, 2014, pp. 27–36. [Online]. Available: http://dl.acm.org/citation.cfm?doid=2666652.2666666

[17] G. Severi, J. Meyer, S. Coull, and A. Oprea, "Explanation-guided backdoor poisoning attacks against malware classifiers," in *30th USENIX Security Symposium (USENIX Security 2021)*, 2021.

[18] B. Tran, J. Li, and A. Madry, "Spectral signatures in backdoor attacks," in *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, ser. NIPS'18. Montréal, Canada: Curran Associates Inc., Dec. 2018, pp. 8011–8021.

[19] B. Chen, W. Carvalho, N. Baracaldo, H. Ludwig, B. Edwards, T. Lee, I. Molloy, and B. Srivastava, "Detecting Backdoor Attacks on Deep Neural Networks by Activation Clustering," *arXiv:1811.03728 [cs, stat]*, Nov. 2018, arXiv: 1811.03728. [Online]. Available: http://arxiv.org/abs/1811.03728

[20] S. Venkatesan, S. Sugrim, R. Izmailov, C.-Y. J. Chiang, R. Chadha, B. Doshi, B. Hoffman, E. A. Newcomb, and N. Buchler, "On detecting manifestation of adversary characteristics," in *MILCOM 2018-2018 IEEE military communications conference (MILCOM)*. IEEE, 2018, pp. 431–437.

[21] M. Melis, A. Demontis, M. Pintor, A. Sotgiu, and B. Biggio, "secml: A python library for secure and explainable machine learning," *arXiv preprint arXiv:1912.10013*, 2019.

[22] A. Chernikova and A. Oprea, "FENCE: Feasible evasion attacks on neural networks in constrained environments," *arXiv:1909.10480*, 2019.

[23] T. Fukuda, M. Suzuki, G. Kurata, S. Thomas, J. Cui, and B. Ramabhadran, "Efficient knowledge distillation from an ensemble of teachers." in *Interspeech*, 2017, pp. 3697–3701.

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 11:09:59 UTC from IEEE Xplore. Restrictions apply.